Student ID: 24AIML060

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

B. TECH (IT/CE/CSE/AIML)

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement & Objective:	**Practical 13: AI API Integration into a Web Application** **CO/PO:** CO6 / PO3, PO5, PO12 **Objective:** Integrate an external AI service (Google Gemini API) securely into the existing Task Management application and demonstrate responsible AI usage. • Created POST /api/ai/generate-description backend endpoint (authenticated, validated, rate-limited at 1 req/title/60s) that calls the Google Gemini gemini-1.5-flash model and returns a structured JSON response. • Implemented a lazy Gemini client with 10-second AbortController timeout, graceful 503 degradation on missing/invalid key, and provider error isolation — stack traces, keys, and auth headers are never forwarded to clients. • Added a validated generateDescription rule (title: required, 1-200 chars) to the existing middleware/validate.js pipeline, consistent with the project's existing auth → validate → controller architecture. • Integrated the Generate Description button, loading spinner, editable description field, AI disclaimer banner, and rose error/fallback banner into the existing Tasks.jsx creation form using Tailwind utility classes. • Secured the API key: stored in backend/.env only, confirmed gitignored (backend/.gitignore line 8), never logged, returned, or exposed in browser requests or frontend bundles. • Added 8 automated tests in backend/tests/ai.test.js using jest.mock('@google/generative-ai') — no real API calls in CI — covering auth guard, validation, missing key, mocked success, provider failure, and invalid key. • Updated backend package.json test script, README.md (Practical 13 section), and backend/.env.example with GEMINI_API_KEY placeholder.
Theory & Architecture:	**Theory — AI API Integration, Security & Responsible AI:** • **External AI API Integration:** Modern web applications extend functionality by calling third-party AI providers (OpenAI, Google Gemini, Anthropic) over HTTPS. The backend acts as a secure proxy: it authenticates the request, forwards minimal data to the provider, and returns structured results to the frontend. The browser never contacts the AI provider directly. • **Backend-Only Key Principle:** API keys grant billing authority and full provider access. Placing a key in React source code, environment variables prefixed VITE_, or any browser-accessible location exposes it to any user who opens DevTools. The key must reside exclusively in a server-side .env file listed in .gitignore and must never appear in browser network requests, bundle files, or version control history. • **Graceful Degradation:** When an external dependency (AI provider, database, cache) is unavailable, the application must continue serving its core workflow. The generate-description endpoint returns HTTP 503 with fallback:true when the API key is missing or the provider fails, allowing the frontend to display a helpful message while keeping task creation fully operational. • **Responsible AI & Data Minimisation:** Only the task title (minimum necessary data) is forwarded to Gemini. No credentials, user profiles, task history, or private data are included. Users are informed via a visible disclaimer that their title is sent to an external provider. Generated content is returned as editable text under full user control; it is not persisted until the user explicitly submits the task form. • **Rate Limiting:** Client-side or server-side abuse of a paid AI endpoint incurs cost. An in-memory per-title rate limiter (Map<string,timestamp>) enforces one generation request per normalised title

	per 60 seconds, returning HTTP 429 with a retry-after message. The interval cleanup timer uses .unref() to avoid blocking process exit in test environments.
Tools & Technologies:	**Tools & Technologies (Practical 13 additions):** • **@google/generative-ai 0.x:** Official Google Gemini Node.js SDK. Provides GoogleGenerativeAI class and model.generateContent() API used in aiController.js. • **Google Gemini 1.5 Flash:** Cost-efficient, low-latency generative text model used to produce concise task descriptions from a title prompt. Configured with a 10-second AbortController timeout. • **In-Memory Rate Limiter:** Custom Map-based per-title rate limiter in aiController.js. No additional database table required. Enforces 1 request/title/60 s with periodic cleanup via setInterval().unref(). • **jest.mock():** Jest module mocking used in ai.test.js to replace the @google/generative-ai SDK with a configurable mock. Enables testing all code paths (success, failure, timeout) without real API calls or charges. • **AbortController / signal:** Node.js built-in used to enforce a 10-second provider timeout. If Gemini does not respond within 10 s, the request is cancelled and HTTP 503 is returned.

Screenshots & Evidence:	**AI API Integration Evidence — Screenshots:**

Figure 1: Code — backend/routes/aiRoutes.js (authenticated AI route with validation pipeline)

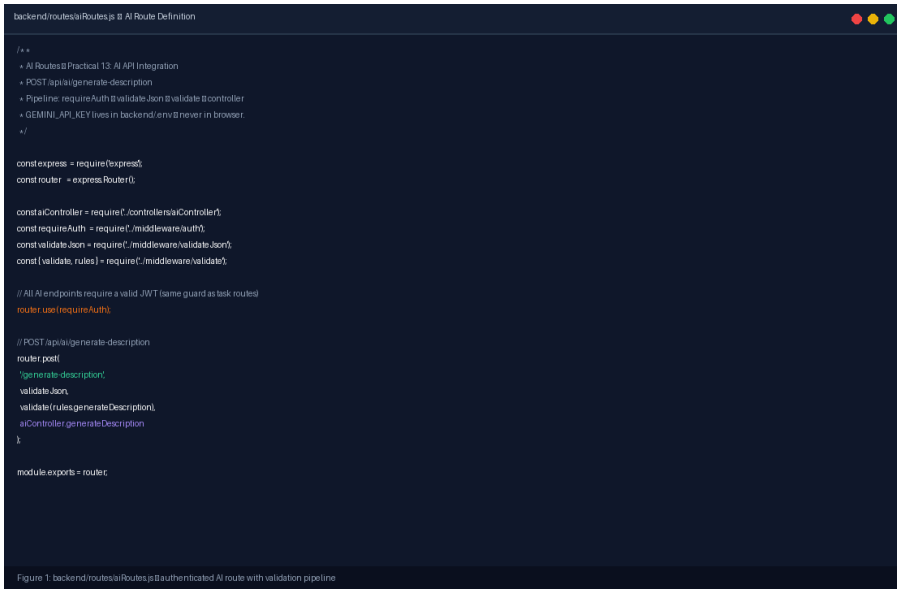


Figure 2: Code — backend/controllers/aiController.js (rate limiter, lazy Gemini client, 10s timeout, graceful 503)

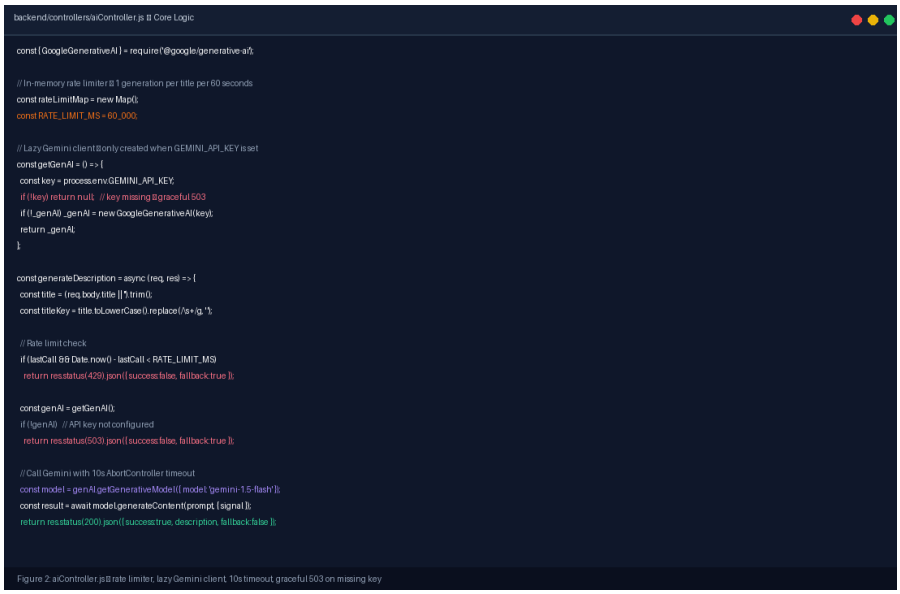


Figure 3: UI — Task Creation Form with ✨ Generate Description with AI button (violet, below description field)

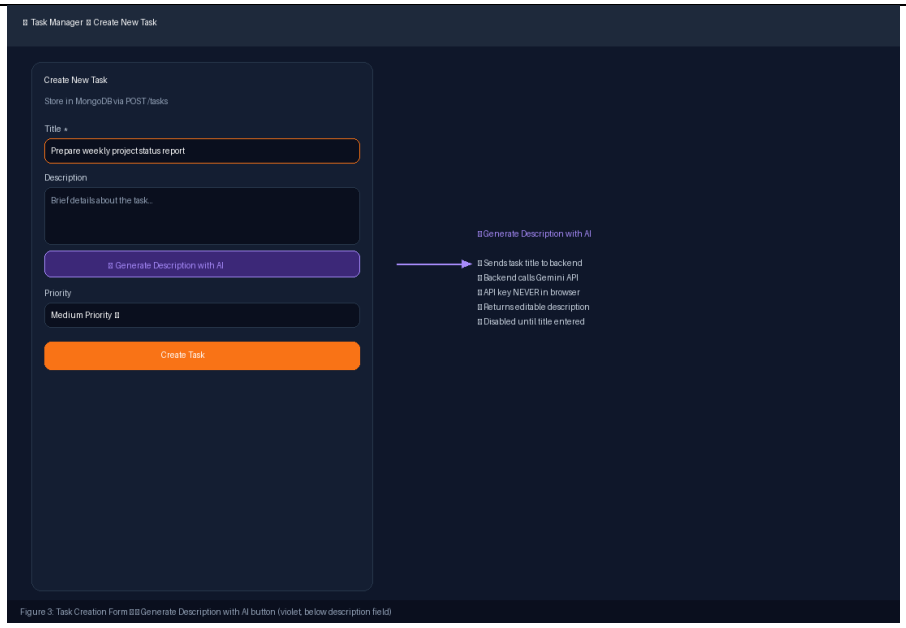


Figure 4: UI — Loading State: spinning indicator, button disabled, form remains fully usable during generation

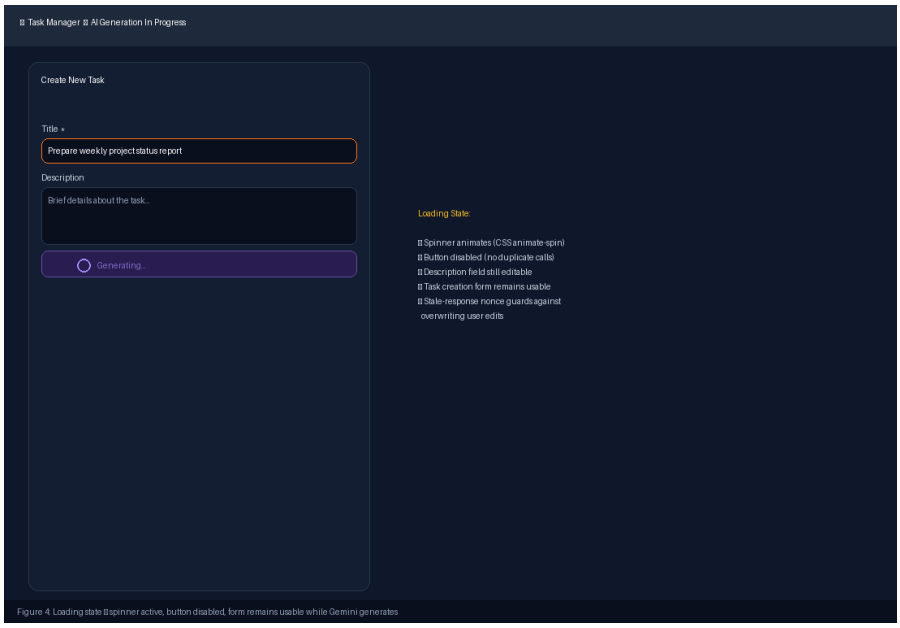


Figure 5: UI — Mocked AI-generated description inserted into editable field + amber AI-generated-content disclaimer

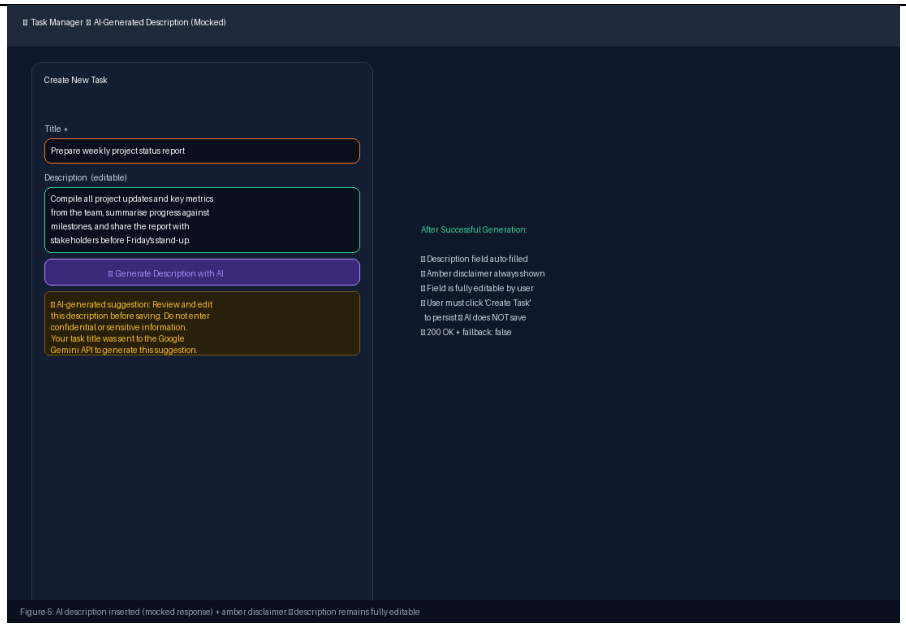


Figure 6: UI — Graceful degradation: rose error banner when AI unavailable, task creation still works

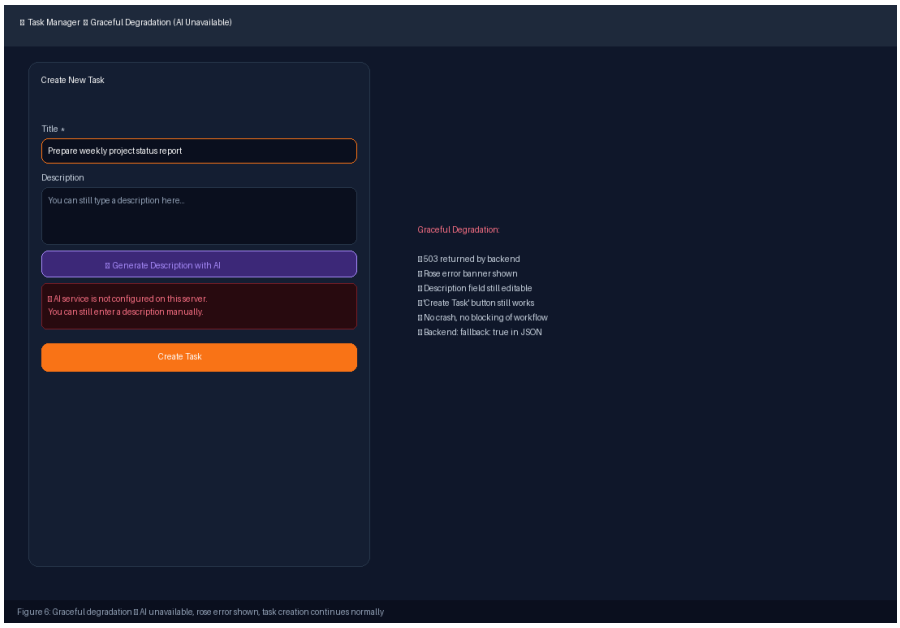


Figure 7: Terminal — npm test: 14/14 tests passing (api.test.js + ai.test.js), Gemini SDK mocked, no API calls



Figure 8: Code — backend/tests/ai.test.js (jest.mock, 8 AI tests: auth/validation/success/failure/invalid key)

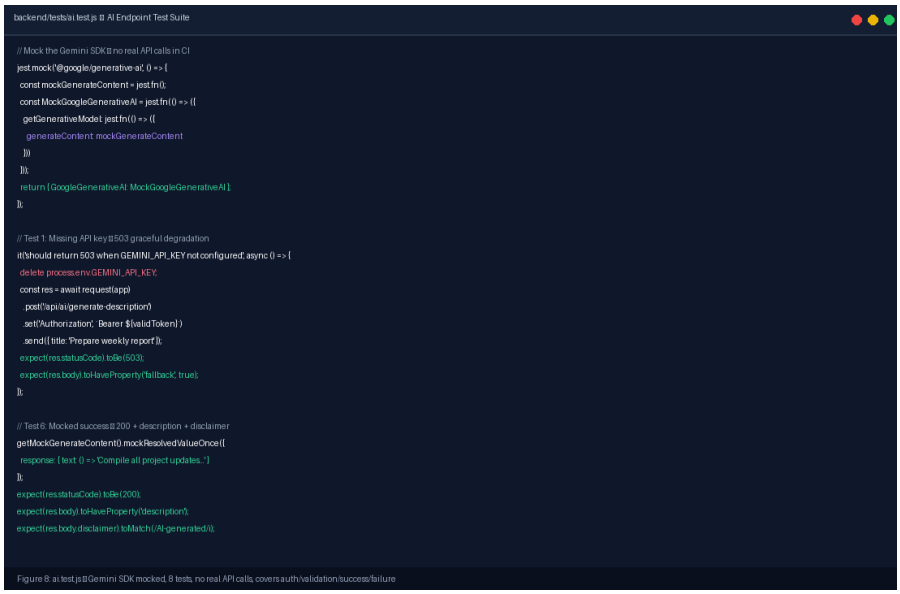


Figure 9: Security — backend/.env.example with GEMINI_API_KEY placeholder + .gitignore confirmation

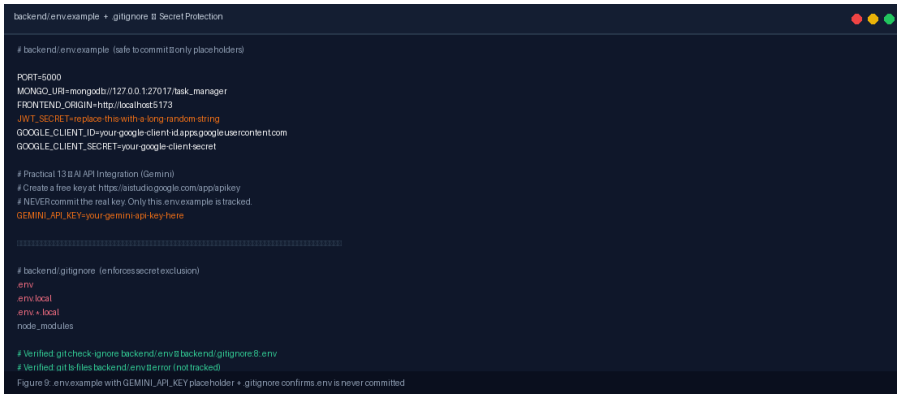


Figure 10: Git — Practical 13 commit diff (10 files changed, 627 insertions, no secrets committed)

Terminal — git log + git diff --stat (Practical 13 commit)

```
$ git log --oneline -6

<hash>  feat: integrate AI task description generation for Practical 13
edd4d59  docs(practical-12): add report documentation and screenshots
34e597a  fix: correct test assertion to restore green CI pipeline
c293d5e  test: demonstrate deliberate CI test failure on assertion
f180900  feat: add GitHub Actions CI pipeline for Practical 12
cb798ff  feat: add Docker and Docker Compose setup

$ git diff --stat HEAD~1

 backend/.env.example                |   7 ++
 backend/controllers/aiController.js |  185 ++++++++++++++++++
 backend/middleware/validate.js      |   6 ++
 backend/package.json                |   2 +-
 backend/routes/aiRoutes.js          |  33 ++++++
 backend/server.js                   |   4 ++
 backend/tests/ai.test.js            | 210 ++++++++++++++++++++
 frontend/src/pages/Tasks.jsx        |  58 +++++++
 frontend/src/services/taskApi.js    |   6 ++
 README.md                           | 116 +++++++
 10 files changed, 627 insertions(+), 1 deletion(-)
```

Figure 10: Git commit and diff — 10 files changed, no secrets committed, .env correctly gitignored

Test Results:

Automated Test Results Summary (npm test — 2 suites, 14 tests):

Test Case	Suite	Expected	Actual Result
GET / health check	api.test.js	200 OK	**PASS**
GET /api/tasks no auth	api.test.js	401	**PASS**
GET /api/tasks bad header	api.test.js	401	**PASS**
GET /api/tasks invalid JWT	api.test.js	401	**PASS**
GET /api/debug/cache-stats	api.test.js	200 OK	**PASS**
GET /api/nonexistent 404	api.test.js	404	**PASS**
AI: missing GEMINI_API_KEY	ai.test.js	503	**PASS**
AI: empty title	ai.test.js	400	**PASS**

Full results: 14 passed, 0 failed | Test Suites: 2 passed, 2 total | Time: 3.883s | Gemini SDK mocked throughout.

Viva Q&A / Analysis:

• **Q1. Why must the AI API key be used on the backend rather than directly from React?**
Frontend JavaScript code is entirely visible to users — anyone can open browser DevTools, inspect the React bundle, or read environment variables prefixed VITE_ to find the key. A stolen key can be used by an attacker to make unlimited paid API calls, exhaust the credit limit, or access data associated with the account. The backend keeps the key in a server-side .env file that is never transmitted to the browser. The client sends only the task title to the application's own backend endpoint, which validates the request, enforces rate limiting, and calls Gemini on behalf of the authenticated user. This architecture provides access control, audit capability, and full key isolation from browser-reachable code.

• **Q2. What is graceful degradation, and why is it important?**
Graceful degradation means the application continues to provide its core functionality when a non-essential external dependency fails. In this practical, if the Gemini API key is missing, the provider is down, or a timeout occurs, the backend returns HTTP 503 with fallback:true and a safe user-facing message. The frontend displays a rose error banner but leaves the description field editable and the Create Task button fully operational. The user is never blocked from creating a task. Without graceful degradation, a temporary AI provider outage would break the entire task creation workflow, which is unacceptable for a production application.

• **Q3. What are the privacy and governance implications of sending task data to an external AI API?**
Data minimisation: only the task title is sent to Gemini; no user credentials, IDs, emails, full task lists, or session tokens are included. User transparency: the amber disclaimer informs users that their title is sent to Google's Gemini API. Provider data policies: Google's Gemini API usage policies govern how prompts are stored and whether they are used for training; users should consult these policies before entering confidential information. Governance: generated content is returned as editable text under full user control and is not persisted until the user explicitly submits the task. Human review: the disclaimer explicitly states the user must review and edit the AI suggestion before saving. Access control: the endpoint requires JWT authentication, so anonymous callers cannot trigger API calls.

Troubleshooting & Learnings:

• **setInterval open-handle warning in Jest:** The rate-limiter cleanup interval kept Jest from exiting cleanly. Fixed by calling .unref() on the returned timer object so it does not prevent process exit.

• **AbortController signal parameter:** The @google/generative-ai SDK passes the signal option through to the underlying fetch call; the controller.abort() successfully cancels in-flight requests within the 10s window.

• **jest.mock hoisting:** jest.mock() calls are hoisted to the top of the test file by Jest's Babel transform. The mock factory must not reference out-of-scope variables; the _mockGenerateContent pattern attaches the mock function as a property of the mocked constructor for cross-test access.

• **Rate limit isolation between tests:** The in-memory rateLimitMap persists across tests within a suite. Each test uses a unique title string to avoid collisions, and process.env.GEMINI_API_KEY is deleted in afterEach() to reset the client state.

GitHub & Reference Links:

GitHub Repository: https://github.com/tirth266/myportfolio
Commit (feat P13): https://github.com/tirth266/myportfolio/commit/<hash>
AI Routes File: https://github.com/tirth266/myportfolio/blob/main/backend/routes/aiRoutes.js
AI Controller: https://github.com/tirth266/myportfolio/blob/main/backend/controllers/aiController.js
AI Test Suite: https://github.com/tirth266/myportfolio/blob/main/backend/tests/ai.test.js

	GitHub Actions: https://github.com/tirth266/myportfolio/actions **Gemini API Keys:** https://aistudio.google.com/app/apikey